Towards an Architecture for Customizable Drones

Mahmoud Hussein[1,2] and Réda Nouacer[1]
[1]CEA, LIST, Software and System Engineering Department (DILS),
P.C. 174, Gif-sur-Yvette, 91191, France
[2]Faculty of Computers and Information, Menofia University, Egypt
mahmoud.hussein and reda.nouacer @cea.fr, mahmoud.hussein@ci.menofia.edu.eg

Abstract— **Drones/UAVs are able to carry out air operations which are difficult to be performed by manned aircrafts. In addition, their use brings significant economic savings and environmental benefits, while reducing risks to human life. However, current generation of embedded drone architectures consist of loosely coupled sub-systems that run in dedicated monolithic platforms. Therefore, this approach reaches its limits when the demand of autonomy increases and when the weight, volume, and power consumption of the dedicated sub-systems encounter the drone restrictions. To overcome these limitations, we, in this paper, propose a modular architectural approach inspired from the very famous *three-layered architecture* within the mobile robots' domain. Our architecture separates the different system's concerns into three layers: *control, flight management,* and *planning*. Thus, each architecture layer can be evolved/customized separately without affecting the other layers. To ensure the applicability of our architecture, we have applied it to a drone system used for a rescue scenario.**

Keywords- Drones/UAVs, System Architecture; Customization; Modularity; Flexibility.

I. INTRODUCTION

Unmanned Aircraft Vehicles (UAVs), which referred to commonly as drones, are air vehicles and associated equipment that fly autonomously or remotely controlled where it does not have a human operator [1]. Drones are increasingly used in civilian and commercial applications such as surveillance, constructions monitoring, agriculture, etc. They can carry out air operations which are difficult for the manned aircrafts. In addition, their use brings significant economic saving and environmental benefits while reducing the risk to human life [2]. Recently, drones have made already the leap to the leisure market, and reached millions of sales. Therefore, evolution on technology, regulations, and society acceptance favour an accelerated deployment of drones for professional applications. According to a 2017 study performed by the Teal Group [3], the world civil drone production is forecasted to make up to $73.5 billion in the next decade, rising from $2.8 billion worldwide in 2017 to $11.8 billion in 2026 (i.e. a 15.5 % compound annual growth rate in constant dollars).

Current embedded architectures for drones (e.g. Paparazzi [4], ICAROUS [5], Open Architecture [6], JHU APL [7], CSAT [8], and MIT/LL [9]) are organized in loosely coupled monolithic boards, which they are composed of memory, processor, and communication resources (e.g. flight planning, flight control, and vision boards). This separation ensures that the subsystems operate almost independently from each other and avoids interference coming from the other parts. However, such approach does not support the continuous development of new drone applications such as the ever-increasing demand on autonomy. Such demand must be met by drone's embedded platforms, while still providing safety, robustness and a small footprint in the physical size and power consumption. Another drawback of the existing architectures is that a huge number of different resources are required for the integration, customization, and maintenance of the drone systems.

In this paper, and in the context of Comp4Drones project, we face the challenge described above through developing a compositional and integrated drone embedded architecture following the famous *three-layered architecture* in the mobile robots' domain [10]. Our proposed architecture separates the different concerns of the system into three layers: *control*, *flight management*, and *planning*. Therefore, each architecture layer can be easily evolved/customized separately without affecting the other architecture layers.

First, the *control layer* contains the low-level control modes of the UAV (e.g. takeoff, landing, trajectory following, etc.). These control behaviors are realized through actuators in case of they are performing an action (e.g. takeoff), or using sensors in the case of perceptional tasks (e.g. capturing video).

Second, the *management layer* is responsible for selecting a plan (generated automatically or designed) in response to an environment situation propagated from the control layer. It is also responsible for executing the selected plans by sending commands to control layer to carry out tasks such as scan area, fly to, etc. Furthermore, in case of, there is no plan exists to cope with the environment situation, a request is sent to the planning layer to provide a new plan.

Third, the *planning layer* is in charge of the high-level planning. Its main task is to generate/provide the different plans that makes the drone system is able to satisfy its missions (requirements). This layer is in charge of task planning, motion planning, diagnosis, monitoring, etc. It also monitors the execution of the plans to ensure safe system operation.

Finally, to ensure the applicability of our architecture, we have applied it to a drone system used for a rescue scenario. For each architecture layer, we specify the basic components that form this layer. For example, the planning layer contains the planning algorithms, and a monitor for plan executions, while the management layer contains the plan executor.

This work was partially funded by ECSEL JU program as a part of the Comp4Drones project with grant number "826610".

978-1-7281-7303-0/20/$31.00 ©2020 IEEE
DOI 10.1109/COMPSAC48688.2020.00019

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.

The remaining of the paper is organized into four sections. Section 2 reviews the existing drone architectures and analyse them. In Section 3, we present our proposed architecture that enables easy customization of the drone system. An example use case following our architecture is presented in Section 4. Finally, conclusions and future work are put in Section 5.

II. Related Work

Recently, a number of architectures for drone systems has been introduced such as Paparazzi [4], ICAROUS [5], Open Architecture [6], JHU APL [7], CSAT [8], and MIT/LL [9]. In following, we briefly describe these architectures.

In order to maximum flexibility, the Paparazzi system was introduced from the start as a distributed architecture [4]. The system is designed to enable the simultaneous operation of multiple vehicles and ground stations. In this architecture, the server agent includes the configuration of each drone (i.e. flight plan, settings, etc.) and additional data (e.g. maps, topography, etc.). The aircraft messages are processed by the server and stored for replaying or analysis. The server can also dispatch asynchronous messages to ground control stations.

In the spirit of SOA, the ICAROUS architecture has been proposed [5]. In this architecture, there is a number of services for providing various drone capabilities (e.g. path planning, geofence containment, sense and avoid, tasks planning, etc.). These services are used commonly for constructing complex UAV applications. The architecture also defines a framework to perform conflict monitoring and its resolutions. Furthermore, for a safe drone operation, conflict monitors and resolvers are coordinated using a decision-making application.

An open architecture for integrating navigation, sensing, and control capabilities of drones into a single unified system has been proposed by researchers in Europe [6]. It is based on the SOA approach and it includes a UAV Service Abstraction Layer (USAL). This layer enables the control of the flight systems and avionics through its high-level services. In this architecture, each major component of the UAV system is encapsulated by a software service container, and then it could become a module of the open architecture.

To demonstrate cooperative behaviours for accomplishing surveillance missions [7], Johns Hopkins University Applied Physics Laboratory (JHU APL) has introduced an autonomous UAV. A part from this work is the development of architecture for such type of systems. The JHU APL architecture consists of two control levels, which provided by separate modules: Low-level control, and high-level mission control. It also contains a wireless communication layer to enable the communications with other UAVs and human pilots at ground control stations (GCSs). The JHU APL architecture is developed for supporting the operation of small drones. It is also considered as a first-generation architecture that has been developed in early 2000s. The architecture is not based on SOA concepts and this could be because of the simple and relatively small number of payloads, which the drone can carry. It also depends on a single communications payload.

UAVs that are capable of co-operative search, acquisition, and track (CSAT) tasks have been developed by MIT and Aurora Flight Sciences [8]. The primary objective of this work is the handling of cooperative search and track functions using better algorithms. However, this work also introduced a high-level functional architecture of drone systems. This architecture includes a control module to plan and control of the UAV. This control module includes three main elements: planning module, vision module, and auto-pilot module. Cooperative behavior between UAVs is enabled through the information sharing.

An architecture for drone's data and software services has been developed in MIT Lincoln Laboratory (MIT/LL) [9]. This architecture is an SOA-based with open standards used for key interfaces. A key feature of the developed architecture is the ability to accommodate sense and avoid capabilities through leveraging ground-based surveillance assets as a short-term goal. In addition, in the long-term, airborne-based sense and avoid capability thar is used for avoiding collisions with other aircraft is also accommodated in the architecture. Furthermore, the services in this architecture are divided into two layers based on their real time performance requirements.

The Future Airborne Capability Environment (FACE) [11] consortium has been formed for defining an open avionics environment for all airborne platforms. They have proposed a reference architecture that is composed of the following logical segments: operating system, input/output, platform-specific services, transport services, and portable components. The architecture also defines standardized interfaces that provide connections between the architectural segments. Furthermore, the architecture has three profiles: security, general purpose, and safety.

Similar to the above architectures, there were efforts in architectures for large and medium-sized aircraft. In the early 1990s, basic internal architectures of large and medium-sized aircraft avionics were based on federated architectures, where each functionality of the aircraft, for instance flight control or navigation, was implemented by dedicated hardware and software. This was the easiest solution to isolate any potential failure, so as not to affect other features [12]. However, with the increase in the number of functions required for aircraft, the federated architectures presented scalability problems [13], which triggered a shift towards Integrated Modular Avionics (IMA) architecture [14].

The IMA architecture uses innovative technical solutions that allow various functionalities of the aircraft to share common hardware (processing modules, network, etc.) while maintaining the same safety/integrity levels provided by the physical isolation of the federated architecture. Since then, IMA-based architectures have evolved towards increasingly distributed setups in which the I/O tasks are transferred from the centralized processing modules usually placed at the aircraft electronics bay, into decentralized elements called remote data concentrators (RDC). The RDCs connect to each other and to the processing modules through a high-capacity deterministic avionics network. With this modification, it was possible to further optimize the

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.

architecture by placing the RDCs in the vicinity of the sensors and actuators of the aircraft, reducing the amount of cabling used.

In summary, the current embedded architectures of drones are organized in loosely coupled monolithic boards. Each board is composed of a memory, a processor, and communication resources (e.g. planning boards, flight control, etc.). This separation ensures that the subsystems operate almost independently from each other and avoids interference coming from the other parts. However, such approach does not support the continuous development of drone applications such as the ever-increasing demand on autonomy. Drone embedded platforms must meet this demand, while still providing safety, robustness, and a small footprint in physical size and power consumption. Another drawback of existing architectures is that there is a huge number of different resources are required to integrate, customize, and maintain the drone systems. Therefore, *there is a need for an architecture to enable easy integration and customization of drone system*s.

III. An Architecture for Customizable Drones

There has been a lot of efforts regarding main conceptual components that are required in architectures for mobile robots. Among existing architectures, the architecture proposed by Gat is one of the clearest representations [10]. This architecture is a transition from sense-plan-act architectures to what is known as three-layered architecture. This architecture arises from the observation that algorithms to control the mobile robots usually fall into three categories. First, algorithms that map sensors directly into actuators without taking into account the internal state in range of milliseconds (i.e. reactive control). Second, algorithms for executing routine sequences of activity that rely on internal system state without performing search in range of seconds. Third, search-based algorithms which is time-consuming depending on the rate of environment changes. Such algorithms have decision cycle in range of seconds or even minutes. Furthermore, each of these algorithms need interfaces to each other for implementing transitions between the different architecture layers.

Based on this three-layered architecture and to enable easy integration and customization of future drone systems, we have proposed an architecture for customizable UAV systems as shown in Figure 1. This architecture separates the different concerns of the drone's system into three layers: *control*, *flight management*, and *planning*. The separation of concerns enables the customization of each layer without affecting the others, where the layers are connected through well-defined interfaces. In the following, we describe these layers, in detail, and in next section, we give an example instance of this architecture.

A. The Planning Layer

The top layer of the UAV system architecture concerns the generation of plans from the missions that need to be supported by the drone (e.g. task planning, motion planning, diagnosis, monitoring, etc.). These plans specify the behavior of drone's system in terms of tasks/actions that lead from an initial system state to the system mission completion. At runtime, a plan tasks are either executed till completion or reconfiguration/re-planning is requested when an environment/system situation is detected that cannot be handled by current plan (i.e. the plans are reactive).

Unlike a sequential plan [15], a reactive plan is a plan that takes into account environment changes by specifying actions towards a given goal (mission) for each state from which that mission is reachable. The plan execution is proceeded by determining the current environment state, choosing the action specified by the plan for that state, performing the action and then determine the new state. Through covering all states from which the mission completion is reachable, it does not matter that a new state reached from an action is the "expected" one or not. As long as, the drone mission can be completed from the new state, and the execution of the plan can continue.

The planning layer is time-consuming which may include search-based algorithms with exponential execution time or with polynomial-time that has large constants (e.g. certain image processing techniques that runs on limited computing resources). A key feature of the planning layer which should be considered is that the system/environment state may change between the time a planning algorithm is invoked and the time when its results has been produced. The planning layer interacts with the rest of the system in two ways: producing plans for the management layer to execute, or responding to specific queries from the flight management layer.

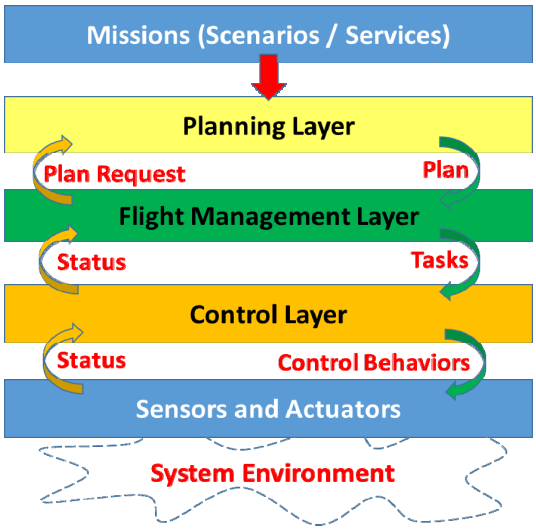

Figure 1 : A layered architecture of UAV System

B. The Flight Management Layer

The *management layer* is in charge of selecting a plan (designed/generated) in response to an environment situation coming from the control layer. The plan can be also propagated from the planning layer to achieve a specific drone mission. This layer is also able to execute the selected/propagated plans by sending set of commands to the control layer. Furthermore, in case of, there is no plan exists to cope with an environment or a system situation, a request is sent to the planning layer to provide a new plan.

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.

To execute a plan's tasks (e.g. fly to, scan area, locate victim, etc.), the management layer is in charge of selecting which control/primitive behavior(s) from the control layer that should be executed at a certain time to achieve the tasks. The challenging problem is that the outcome of selecting a particular control behavior for a particular situation might not be the intended regarding current system state. An approach for solving this problem is to enumerate all the possible system states, and then specify the correct control behavior to use in each system state to achieve a particular task. However, this approach is often not possible because of the large number of states the drone can be in combined with the different tasks. Therefore, an alternative solution is to use conditional sequencing. This approach is motivated by human instruction following [16].

C. *The Control Layer*

The control layer consists of control behaviors (functions) that implement the feedback control loops connecting sensors to actuators or realize tasks of a drone's mission. The functions are activated at any given time using external input coming to the control layer from the management layer or in response to environment changes (i.e. propagated from the sensors). A number of examples for such primitive/control behaviors are take-off, landing, cruising, etc. The behaviors are realized by actuators in case of they are performing an action, or through sensors in the case of perceptional tasks.

The control layer functions have a number of constraints. First, they should have a bounded time and space to react as fast as possible. Second, the functions should be able to detect any failure to perform the desired behavior. This allows the system management and planning layers to recover from the failure by taking corrective actions. Third, internal state of the control layer should not make discontinuities in a function, where the flight management layer needs to manage transitions between regimes of continuous functions.

IV. EVALUATION

In order to demonstrate our architecture's applicability, we have used it to specify a drone system for a rescue scenario. In the following, we describe, in detail, the rescue scenario and some of the architecture components of the drone's system to achieve the missions of this scenario.

A. *Rescue Scenario*

When earthquake and/or tsunami occurs, the highest priorities is to look for survivors in isolated areas where roads become inaccessible, and to provide a relief through delivering medical supplies, food, and water. In such situations and when there is a shortage of trained helicopter and fixed-wing pilots, a fleet of co-operative drones with ground operation facilities can be used to scan for injured humans and then deliver a relief packages that contains the food, the water, and the needed medical supplies.

The above scenario poses two high level goals (missions) that need to be achieved: (a) a specific area needs to be scanned by UAVs for generating a saliency map that consists of geolocations of injured persons; and (b) based on the saliency map and the available drones, a logistics plan needs to be put for delivering medical and food supplies to the identified locations within this area.

In an ideal situation, a person would like to specify a drone mission at the above described high level of abstraction. Then, the UAVs in collaboration generate the appropriate plans based on their availability, resources, and capabilities using sophisticated planning techniques for teams of UAVs and human resources in a mixed manner. The tasks of these plans should be clear and specific enough to be executed. To execute a task, a number of low-level control behaviors for each UAV in the fleet are executed and coordinated in a distributed manner. An important aspect of the above process is that a mission is specified at different levels of abstraction (i.e. plan, tasks, and control behaviors) which require a high degree of autonomy to transit from one level to another.

In the following sub-sections, we describe how our proposed architecture can be used to design a drone system that is able to achieve the rescue mission goals. It supports the specification of a mission at high level. Then, this plan can be translated to tasks, and low-level control behaviors of the drone system.

B. *Planning Layer*

To support the rescue scenario, the planning layers needs to contain two main components: task planners and a plan monitor. First, the task planners are used to compose high-level actions (e.g. "take-off", "fly-to point A," etc.) into plans that satisfies a set of declaratively specified missions/goals (e.g. acquiring images of certain area, or delivering boxes of emergency supplies to certain places after a natural disaster). Example planners that can be used in this layer are TALplanner [17] and TFPOP [18]. The results of the planners are plans in form of task specification tree [19]. A task in this tree is often interpreted as an action or a number of actions (action composition) that are combined through standard control structures such as concurrency, sequence, and conditional branching. An example part of a plan is shown in Figure 2. This plan consists of a set of tasks such as scan area, transmit data, fly-to destination. The scan area and transmit data are performed concurrently while the fly to destination is perform after the concurrent tasks are completed.

Second, regardless of how much effort spent on specifying all possible contingencies, the plans may still fail. Therefore, the execution of a plan needs to constantly monitored using an execution monitor to detect and recover from failures. For example, a drone may drop its cargo before reaching destination, and then the drone should monitor a condition that specifies the cargo remains until the destination is reached. Another example is that the carrier can be heavy, and then the UAV should be able to detect takeoff failures. Such examples are called monitoring conditions. To ensure that the monitoring conditions hold during the plan execution, information about the drone environment is collected at a given frequency. This information forms the new environment state, which is then used to evaluate the monitoring conditions [20].

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.

C. Management Layer

The management layer provides transition between tasks specified in the planning layer and control behaviors of the control layer. It has the tasks execution engine which has two interfaces with both the control and management layers (see Figure 2). In the following, we describe such elements.

First, tasks generally consist of control behaviors that are used sequentially or in parallel. For example, to do a building surveillance, takeoff, landing, path-following, camera control, and other behaviors are used. The switching between these behaviors is not easy and requires a smooth handover. Thus, an execution engine is need to do the switching in an efficient manner. A solution to the switching problem is to use Finite State Machines (FSMs) [21], where control behaviors are modelled as states and transitions between them is guarded by conditions, and actions. In addition, in case of a complex system, it can be modelled at different levels of abstraction using Hierarchical Concurrent State Machines (HCSMs) [22]. These machines can be easily executed, where they have well defined semantics.

Second, the interface between the execution engine and the control layer can be performed through command languages, where for each control behavior, there is a command to call it. These commands can be either control (e.g. climb (altitude), traj3d_ flyto (longitude, latitude, altitude, velocity), etc.), or payload and perceptional (e.g. zoom (value), laser_angular_resolution (value), etc.).

Third, the interface between the execution engine and the management layer is done through either requesting a plan in form of a task specification tree or the plan is sent directly to the engine to achieve a certain goal/mission.

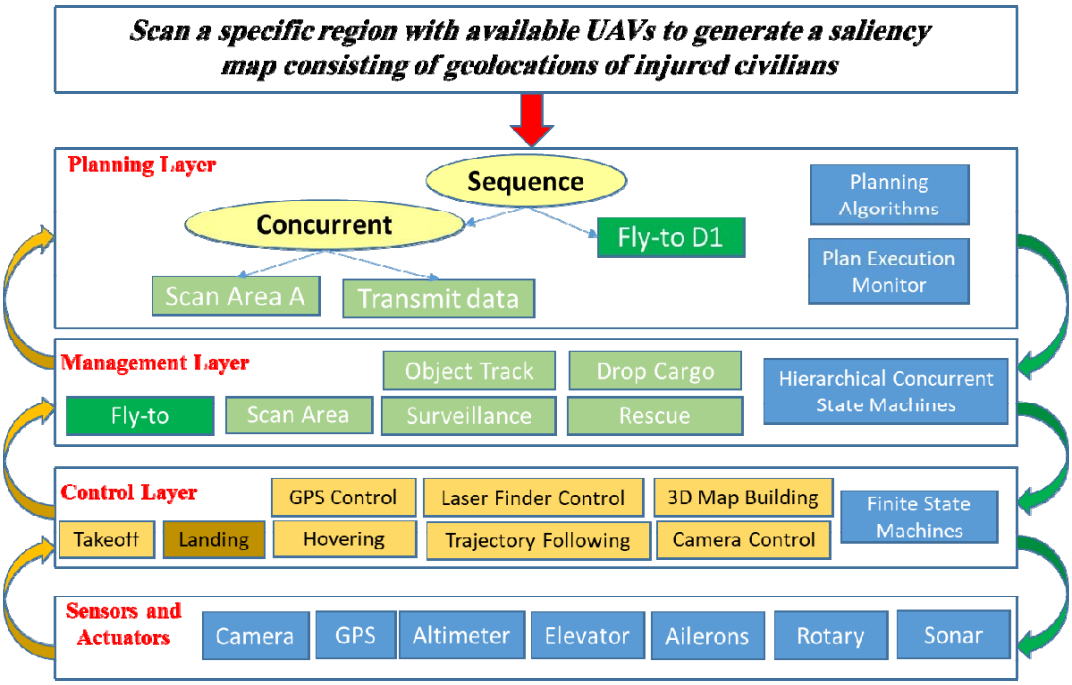

Figure 2 : An example of UAV different layers

D. Control Layer

The control layer encapsulates all core functions that implement the basic flight and payload behaviors as well as functions for perception capabilities that is used by planning and management layers (see Figure 2). In the following, we describe some examples of these behaviors.

First, the control layer has flight control behaviors such as: (a) hovering to keep the drone in a desired position, altitude, and heading based on current sensor readings; (b) takeoff to bring the UAV from the ground to a desired altitude; (c) path-following to fly between specified waypoints of a flight path; and (d) vision-based landing to land the drone relative to a state estimation performed onboard without the GPS.

Second, to realize the rescue scenario, the control layer includes a number of control behaviors for payloads such as: (a) visual servoing control for pan-tilt camera to keep an object in the middle of the image taken by the camera; (b) a GPS coordinate control for pan-tilt camera to keep a geographical location centered in the camera image; (c) control for camera parameters such as zoom, shutter, and iris; (d) control for laser range finder parameters such as measurement angular resolution and distance resolution.

Third, the control layer contains behavior for perception functionalities which include: (a) vision-based state estimation that provides an accurate pose estimated in relation to a designed pattern for the autonomous vision-based landing; (b) vision-based object tracking to provide an estimate for the position relative to a moving object; (c) 3D map building using laser range finder which provides a 3D map of the environment using the laser range finder data; (d) human body identification by identifying position of a human body in a stream of thermal/colored images.

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.

E. Discussion

One of the main features of our proposed architecture is the ability to be easily customized/adapted for a certain mission. This can be achieved through localized changes that can be done to each of the architecture layer without affecting the other layers, where they are linked with well-defined interfaces. For example, to make the drone described above able to execute a new mission. First, the control layer is going to be changed by adding new control behaviors for the drones' payload (e.g. a mechanism to control the rotation of a laser range finder), and the drone perceptions (e.g. drone localization using reference images to provide an absolute position based on geo-referenced images when GPS is denied in the environment). Second, new tasks that make use of the new control behaviors are defined at the management layer. In addition, new commands are specified to enable the executing of the new control behaviors. Finally, a new plan(s) that is able to achieve the new mission has to be either designed or automatically generated using planning algorithms at the planning layer.

V. Conclusions and Future Work

Current embedded architectures for drones are organized in loosely coupled monolithic boards. Such approach does not support the continuous development of drone applications such as the ever-increasing demand on autonomy where costs for integration, customization and maintenance in terms of component provisioning and handling are very high. In this paper, and in the context of Comp4Drones project, we face this challenge by proposing a compositional and integrated drone embedded reference architecture. Our architecture separates the different concerns of the drone's system into three layers: *control*, *flight management*, and *planning*. The separation of concerns enables the customization of each layer without affecting the others, where these layers are connected through well-defined interfaces. To demonstrate our architecture's applicability, we have used it to specify a drone system used for a rescue scenario.

In the future, and during the Comp4Drones project, we are planning to apply our architecture to a number of case studies to ensure its applicability. In addition, we are going to implement this architecture and use it in real world scenarios with existing drones.

References

[1] SESAR JU, "European ATM Master Plan: Roadmap for the safe integration of drones into all classes of airspace," 2017. [Online]. Available: https://www.sesarju.eu/sites/default/files/documents/reports/European%20ATM%20Master%20Plan%20Drone%20roadmap.pdf.

[2] SESAR JU, "Report: European Drones Outlook Study," 2016. [Online]. Available: https://www.sesarju.eu/sites/default/files/documents/reports/European_Drones_Outlook_Study_2016.pdf.

[3] McKinsey analysis; Teal Group, "World Civil Unmanned Aerial Systems Market Profile & Forecast," 2017.

[4] G. Hattenberger, M. Bronz, and M. Gorraz, "Using the Paparazzi UAVSystem for Scientific Research," in International Micro Air VehicleConference and Competition (IMAV), Delft, Netherlands, 2014.

[5] Consiglio, M., Munoz, C., "ICAROUS: Integrated Configurable Algorithms for Reliable Operations of Unmanned System," in IEEE Digital Avionics Systems Conference, Sacramento, CA - September 25-29, 2016.

[6] E. Pastor, C. Barrado, P. Royo, J. Lopez, and E. Santamari, "An Open Architecture for the Integration of UAV Civil Applications," in Thanh Mung, ed., Aerial Vehicles, 2009.

[7] Robert J. Bamberger, Jr., David P. Watson, David H. Scheidt, and Kevin L. Moore, "Flight Demonstration of Unmanned Aerial Vehicle Swarming Concepts," Johns Hopkins APL Technical Digest, vol. 27, n° %11, 2006.

[8] J. How, C. Frasher, K. C. Kulling, L. F. Bertuccelli, O. Toupet, L. Brunet, A. Bachrach, and N. Roy, "Increasing Autonomy of UAVs," IEEE Robotics & Automation Magazine, 2009.

[9] Curtis W. Heisey et. al., "A Reference Software Architecture to Support Unmanned Aircraft Integration in the National Airpsace System," Journal of Intelligent Robotic Systems, 2012.

[10] E. Gat, "On Three-layer architectures", in Artificial Intelligence and Mobile Robots, David Kortenkamp,R. Peter Bonnasso, and Robin Murphy, eds., AAAI Press, MA, USA, p. 195–210, 1998.

[11] S. H. VanderLeest, "Designing a future airborne capability environment (FACE) hypervisor for safety and security," in IEEE/AIAA 36th Digital Avionics Systems Conference (DASC), 2017, St. Petersburg, FL, 2017.

[12] Henning Butz, "Open integrated modular avionic (IMA): State of the art and future development road map at Airbus deutschland," Technical report, Department of Avionic Systems at Airbus Deutschland GmbH, 2010.

[13] Christopher B. Watkins and Randy Walter, "Transitioning from federated avionics architectures to integrated modular avionics," in 26th Digital Avionics Systems Conference, Grand Rapids, Michigan, 2007.

[14] Itier, Jean-Bernard, "A380 integrated modular avionics: The history, objectives and challenges of the deployment of IMA on A380," Technical report, AIRBUS S.A.S., 2010.

[15] M. Ghallib, D. Nau, P. Traverso, "Automated Planning: Theory and Practice". Morgan Kaufman, 2005.

[16] Philip E. Agre and David Chapman, "What are plans for?," Robot. Auton. Syst., vol. 6, n° %11, pp. 17-34, 1990.

[17] P. Doherty and J. Kvarnstrom,, "TALplanner: an empirical investigation of a temporal logic-based forward chaining planner," in Proceedings. Sixth International Workshop on Temporal Representation and Reasoning, Orlando, FL, USA, 1999.

[18] J. Kvarnström, "Planning for loosely coupled agents using partial order forward-chaining," in Proceedings of the Twenty-First International Conference on International Conference on Automated Planning and Scheduling (ICAPS'11), AAAI Press, 138–145, 2011.

[19] Doherty P., Heintz F., Landén D., "A Distributed Task Specification Language for Mixed-Initiative Delegation" in PRIMA, Lecture Notes in Computer Science, vol 7, 2012.

[20] F. A. e. al., "A Foundation for Runtime Monitoring," in Lahiri S., Reger G. (eds) Runtime Verification. Lecture Notes in Computer Science, vol 10548. Springer, Cham, 2017.

[21] D. Lee and M. Yannakakis, "Principles and methods of testing finite state machines-a survey," in Proceedings of the IEEE vol. 84, no. 8, pp. 1090-1123, 1986.

[22] Peter M. Kielar, Oliver Handel, Daniel H. Biedermann, André Borrmann, "Concurrent Hierarchical Finite State Machines for Modeling Pedestrian Behavioral Tendencies,," Transportation Research Procedia, vol. 2, pp. 576-584, 2014.

Authorized licensed use limited to: Gran Sasso Science Institute. Downloaded on February 19,2026 at 16:01:10 UTC from IEEE Xplore. Restrictions apply.